REAL-TIME COLLABORATION BACKEND

Individual Work Plan • Person 2 • Assigned to Adarsh

1. Mission

Adarsh owns the real-time collaboration backend for the Real-Time Collaborative IDE & Debugger. His goal is to make multiple users edit the same document in a shared room, with changes synchronized through Yjs and WebSockets and with live participant presence.

Core outcome: two browser sessions join the same room; when either user edits the document, the other session receives the change and both converge to the same document state.

2. Responsibilities

A. Set up the collaboration service

- Create/configure the Node.js collaboration server.
- Install and configure Yjs and y-websocket (or the compatible server setup chosen by the team).
- Define how room IDs are represented and how clients join a room.
- Document the local start command, required ports, and environment variables.

B. Synchronize documents

- Use Yjs as the shared CRDT document model.
- Coordinate with Person 1 to bind the Yjs text type to Monaco using y-monaco.
- Verify bidirectional updates between two browser sessions.
- Test simultaneous edits and ensure clients converge to a consistent state.
- Avoid implementing a second, conflicting text-sync mechanism.

C. Presence and remote cursors

- Use Yjs awareness/presence to track connected participants.
- Share user display name and cursor/selection state needed by the editor.
- Coordinate with Person 1 on the data shape used to render remote cursors and selections.
- Handle users leaving the room so stale presence is cleared.

D. Connection recovery and room access

- Test disconnect/reconnect behavior and resynchronization.
- Validate room access on the server when authentication/access controls are added.
- Do not trust a room ID supplied by the browser as proof of authorization.
- Coordinate with Person 4 on workspace IDs, user identity, and persistence requirements.

E. Integration and testing

- Provide Person 1 with the WebSocket URL, room naming convention, and client connection example.
- Coordinate with Person 4 on saving/restoring document state; Yjs live sync alone is not durable persistence.
- Keep the collaboration service independently runnable and provide a short README.
- Do not own the terminal service, PostgreSQL implementation, or AI debugger; those belong to other team members.

3. Suggested Module Layout

Adapt this to the team's existing repository structure instead of duplicating files.

```
server/
  collaboration/
```

```
    server.js          # WebSocket/Yjs service entry point
    roomConfig.js      # room configuration (optional)
    README.md           # setup and integration notes
src/
  collaboration/
    createYjsProvider.js  # browser provider helper (coordinate with Person 1)
    presence.js            # awareness helpers (optional)
```

4. Suggested Work Sequence (24-hour hackathon)

Time	Tasks	Milestone
Hours 0–2	Confirm room IDs and integration contract with Person 1; initialize collaboration service.	Service starts.
Hours 2–5	Connect Yjs document/provider to a minimal editor test.	Two sessions sync.
Hours 5–8	Integrate y-monaco with Person 1's Monaco editor.	Actual IDE editor syncs.
Hours 8–12	Add awareness data and remote cursor/selection support.	Presence works.
Hours 12–16	Test concurrent edits, disconnect/reconnect, and room separation.	Core cases pass.
Hours 16–20	Coordinate persistence/access-control integration with Person 4.	Integration contract confirmed.
Hours 20–24	Fix bugs, document setup, and run a two-browser demo.	Demo-ready sync.

5. Contracts to Agree on Early

- **Room identity:** agree whether the room ID is the workspace ID or a separate collaboration-room ID.
- **Provider:** document the WebSocket endpoint and the client setup expected by Person 1.
- **Awareness payload:** agree on fields such as display name, user ID, cursor position, and selection range. Share only the data needed for presence.
- **Persistence:** coordinate with Person 4 on how document updates are saved/restored. Do not assume in-memory Yjs state survives a server restart.
- **Access control:** decide how users are authorized to join rooms; enforce authorization on the server.

6. Acceptance Checklist

- ■ Collaboration server starts using documented instructions.
- ■ Two browser sessions can join the same room.
- ■ Edits made in either session appear in the other.
- ■ Simultaneous edits converge to consistent document content.
- ■ Remote cursor/selection presence is visible when integrated with Person 1's UI.
- ■ Presence clears after a user leaves or disconnects.
- ■ Different rooms do not accidentally share document state.
- ■ Reconnect behavior is tested and documented.
- ■ Workspace persistence expectations are agreed with Person 4.
- ■ A two-browser demo works without manual database edits.

7. Important Notes

- Yjs handles conflict-free shared document state; Monaco is the editor UI. Use the agreed binding rather than manually copying text between clients.
- Presence/cursor data is separate from durable file content and should not be treated as saved document data.
- A WebSocket room is not automatically secure. Enforce room membership and authorization before production/public deployment.

- Keep the first milestone narrow: prove two-browser editing first, then add cursors, recovery, and persistence integration.

Definition of done: two users can edit one shared Monaco document in real time, see collaborator presence, and recover cleanly from a tested reconnect flow; setup and integration instructions are documented.